

Lab 1 and Lab 2 Defence Guide: Sorting Complexity and Quicksort Optimisation

Prepared for: DSA coursework review

Repository reviewed: 1

Scope: Lab 1, “Sorting, Complexity,” and the Lab 2-style quicksort improvement work described in the same handout.

Important evidence note. The repository currently contains the lab handout, `Bench.java`, the unmodified `Insertion.java` and `Merge.java`, an incomplete `Quick.java` with all three TODO sections still present, and a template `answers.txt`. It does **not** contain a completed `output.txt`, completed answers, or a committed Lab 2 implementation. Therefore, this guide distinguishes between (a) conclusions that follow from the code and handout, and (b) results that must be obtained from a completed run. Any timing table labelled “supplementary reconstruction” is evidence from a separate controlled run, not a claim that those numbers were submitted in the repository.

1. The central idea: complexity is a growth pattern, not a stopwatch ranking

When the lab asks for the complexity of an algorithm, it is asking how its running time changes as the input size, n , grows. It is not asking which algorithm is fastest for one particular machine or one small array. Hardware, Java warm-up, garbage collection, compiler optimisation, and constant implementation costs can change the absolute time. Complexity deliberately ignores constant factors and focuses on the shape of the growth.

The three patterns used by the lab are:

Complexity	Meaning as n grows	What the “time per item” column tends to show
Linear, $\Theta(n)$	Doubling n roughly doubles total time	Approximately constant
Linearithmic, $\Theta(n \log n)$	Doubling n multiplies time by slightly more than two	Grows slowly, approximately like $\log n$
Quadratic, $\Theta(n^2)$	Doubling n multiplies time by about four	Grows approximately in proportion to n

The handout's "per item" column is especially useful because it divides total time by n . If total time is $c n$, per-item time is approximately c ; if total time is $c n \log n$, per-item time is approximately $c \log n$; and if total time is $c n^2$, per-item time is approximately $c n$.

A useful defence sentence is:

"I classified the complexity from the trend in total time and time per item, not from the algorithm name alone. A constant-looking per-item column indicates linear growth; a slowly increasing per-item column indicates linearithmic growth; and a per-item column increasing roughly with input size indicates quadratic growth."

2. What the benchmark actually measures

Bench.java tests each algorithm on the sizes 10, 30, 100, 300, 1000, 3000, 10000, 30000, 100000. It uses three input categories, represented by the RANDOMNESS values 100, 5, and 0:

Benchmark label	Generator parameter	Interpretation
Random	100	Every generated position takes the random branch
95% sorted	5	About 95% of positions use the nondecreasing branch and about 5% are random
Sorted	0	Every position uses the nondecreasing branch

There is a subtle point worth defending accurately: the generator creates a nondecreasing stream by adding a random offset of 0, 1, or 2, and occasionally inserts a random value. Thus "95% sorted" is a controlled **degree of sortedness**, not a mathematical guarantee that exactly 95% of all element pairs are already in final order. The fully sorted case can also contain duplicate values.

For correctness, Bench.java clones the input, sorts one clone using the candidate algorithm, sorts another with Java's Arrays.sort, and compares the arrays. It tests every size from 0 through 1000 for all three input categories. For timing, it copies inputs before each repetition, targets at least 10 ms of work, performs multiple measurements, and keeps the smallest measured runtime. This reduces—but does not eliminate—noise from allocation, garbage collection, and the operating system.

The benchmark's use of repeated copies is important: the same original input is not accidentally sorted once and then reused as an already-sorted input on the next trial. The algorithm receives a fresh clone each time.

3. Lab 1: understand each algorithm before classifying it

3.1 Insertion sort

The implementation in `Insertion.java` scans from left to right. At iteration `i`, it treats `a[0..i-1]` as sorted, stores `a[i]` in `value`, shifts larger elements one position to the right, and writes `value` into the gap.

Pseudocode equivalent:

Plain Text

```
INSERTION-SORT(A)
  for i = 1 to length(A) - 1
    value = A[i]
    j = i
    while j > 0 and A[j - 1] > value
      A[j] = A[j - 1]
      j = j - 1
    A[j] = value
```

A more intuitive explanation is to imagine sorting playing cards held in your hand. You pick up one new card and slide it left until it reaches its correct position among the cards already held. If the hand is already ordered, each new card needs only one comparison. If the hand is reversed, each new card travels across almost the entire sorted prefix.

The loop invariant is:

Before iteration `i`, the prefix `A[0..i-1]` is sorted and contains exactly the elements originally found there.

For a random array, the average amount of shifting is proportional to `i`, so the total work is proportional to $1 + 2 + \dots + n$, which is $\Theta(n^2)$. For a sorted array, the inner `while` condition fails immediately for each item, so the work is $\Theta(n)$. A partially sorted array sits between these extremes; its time depends on how many inversions it contains. An inversion is a pair (i, j) with $i < j$ but $A[i] > A[j]$. Insertion sort's shifts are directly related to inversions [4](#).

3.2 Top-down merge sort

The implementation in `Merge.java` allocates one auxiliary array and recursively divides the input into halves. It sorts each half and then merges the two sorted halves.

Pseudocode equivalent:

Plain Text

```

MERGE-SORT(A, aux, lo, hi)
    if hi <= lo
        return
    mid = floor((lo + hi) / 2)
    MERGE-SORT(A, aux, lo, mid)
    MERGE-SORT(A, aux, mid + 1, hi)
    MERGE(A, aux, lo, mid, hi)

```

Plain Text

```

MERGE(A, aux, lo, mid, hi)
    copy A[lo..hi] into aux[lo..hi]
    i = lo
    j = mid + 1
    for k = lo to hi
        if left half is exhausted, take aux[j]
        else if right half is exhausted, take aux[i]
        else if aux[j] < aux[i], take aux[j]
        else take aux[i]

```

The intuitive explanation is “sort two piles, then zip them together.” At every level of recursion, all n items are copied/inspected across the merges. The recursion has about $\log n$ levels, so total work is $\Theta(n \log n)$. This remains true for random, 95%-sorted, and sorted input because the supplied implementation always performs the recursive calls and the merge; it does not test whether the two halves are already in order.

Merge sort therefore has a valuable guarantee: its running time is $\Theta(n \log n)$ for every input order, at the cost of $\Theta(n)$ auxiliary storage [3](#).

3.3 Basic quicksort in this repository

The supplied quicksort chooses the first element of the current subarray as the pivot, partitions the subarray around it, and recursively sorts the two resulting subarrays.

Pseudocode equivalent:

Plain Text

```

QUICKSORT(A, lo, hi)
    if hi <= lo
        return
    j = PARTITION(A, lo, hi)
    QUICKSORT(A, lo, j - 1)
    QUICKSORT(A, j + 1, hi)

```

The partition routine maintains two scans. The left scan searches for an element that belongs on the right; the right scan searches for an element that belongs on the left. Those elements are exchanged. Finally, the pivot is exchanged into its final position j .

The intuitive explanation is “choose a representative, put everyone smaller on its left and everyone larger on its right, then repeat.” The quality of the representative is crucial. If the pivot divides the array near the middle each time, the recursion has about $\log n$ levels and each level scans roughly n elements, giving average $\Theta(n \log n)$. If the pivot is repeatedly the smallest or largest item, one side is empty and the other has $n-1$ items. The recurrence becomes approximately $T(n) = T(n-1) + \Theta(n)$, which is $\Theta(n^2)$.

For the repository’s first-element pivot, sorted input is exactly the dangerous case: the first element is the smallest, so the left partition is empty at every step. The 95%-sorted input is also likely to produce poor partitions. Random input usually produces a more balanced sequence of partitions, so the observed trend is approximately linearithmic.

The distinction between **average-case** and **worst-case** must be stated explicitly:

Basic quicksort is not “always $\Theta(n \log n)$.” It is $\Theta(n \log n)$ under sufficiently balanced/average partitions, but $\Theta(n^2)$ in the worst case. The original repository implementation exposes that worst case because it uses the first element as pivot without randomisation.

4. Lab 1 classification table

The expected classification from the benchmark trends is:

Algorithm	Random input	95% sorted input	Sorted input	Reason
Insertion sort	$\Theta(n^2)$	Usually between the extremes; in the lab’s coarse choices, classify from measured trend, commonly quadratic if the 5% random elements create $\Theta(n^2)$ shifts	$\Theta(n)$	Work is proportional to the number of shifts/inversions
Basic quicksort	$\Theta(n \log n)$ average	$\Theta(n^2)$ tendency in this first-element implementation	$\Theta(n^2)$	Sortedness makes the first element a consistently extreme pivot

Top-down merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	It recursively splits and merges regardless of input order
----------------------------	--------------------	--------------------	--------------------	--

The exact insertion-sort label for the 95%-sorted column must be justified from the actual timing table, because “95% sorted” is a generator convention rather than a formal asymptotic family. If the benchmark’s per-item time rises approximately with n , call it quadratic; if it remains nearly constant over the measured range, call it linear-like. Do not claim that every fixed “5% random” generator must have one universal asymptotic label without inspecting the data.

For the repository’s handout, the filled-in answer should therefore normally be:

Plain Text

Insertion.java:

```
random: Quadratic
95% sorted: classify from the timing trend; commonly Quadratic in this lab
sorted: Linear
```

Quick.java:

```
random: Linearithmic
95% sorted: Quadratic
sorted: Quadratic
```

Merge.java:

```
random: Linearithmic
95% sorted: Linearithmic
sorted: Linearithmic
```

A strong defence does not merely recite the table. It says: “Insertion sort is sensitive to inversions, quicksort is sensitive to pivot balance, and this merge sort is insensitive to input order because it always performs the same divide-and-merge structure.”

5. Lab 2: what exactly is being changed?

The three permitted quicksort improvements are independent switches in the constructor:

Java

```
new Quick(shuffleFirst, useMedianOfThree, insertionSortCutoff)
```

The baseline is:

Java

```
new Quick(false, false, 0)
```

This means: do not shuffle, use the first element as pivot, and never switch to insertion sort. The example “all improvements” configuration is:

Java

```
new Quick(true, true, 42)
```

This means: shuffle once before recursion, use median-of-three pivot selection during partitioning, and use insertion sort on sufficiently small recursive subarrays.

5.1 Does “one at a time” mean resetting the previous change?

Yes, for the **isolated-effect experiment**, each improvement should be compared with the same baseline while the other improvements remain disabled:

Test	Configuration	What it answers
Baseline	(false, false, 0)	How bad is the original implementation?
Shuffle only	(true, false, 0)	What does shuffling alone change?
Median-of-three only	(false, true, 0)	What does pivot sampling alone change?
Cutoff only	(false, false, c)	What does the small-subarray switch alone change?

Do **not** leave the shuffle enabled when testing median-of-three if the question is “what did median-of-three alone do?” Otherwise the measured change combines two effects.

After the isolated tests, perform the **cumulative/combination experiment**. Here, previous adjustments are intentionally kept and another one is added:

Plain Text

```
baseline -> best single improvement -> best pair -> best triple
```

For example, test `(true, false, 0)` , `(true, true, 0)` , and `(true, true, c)` . Also test the other pairs, because the best single improvement is not guaranteed to produce the best pair. The lab explicitly asks for combinations, so the correct interpretation is: **reset for one-at-a-time attribution; retain changes when evaluating combinations.**

5.2 Improvement 1: shuffle once before sorting

The implementation should do this only in the public `sort` method:

Plain Text

```
QUICKSORT(A)
    if shuffleFirst
        SHUFFLE(A)          // exactly once
    QUICKSORT(A, 0, n - 1)
```

Shuffling destroys the correlation between the original input order and the pivot rule. A sorted input is converted into a random permutation, so choosing the first element is no longer systematically choosing the minimum. The expected complexity becomes $\Theta(n \log n)$, including on originally sorted or 95%-sorted inputs, although the worst-case theoretical bound remains $\Theta(n^2)$: a random shuffle makes the bad sequence unlikely; it does not make it impossible.

It is important not to shuffle inside every recursive call. That would add unnecessary work, destroy the useful random-subarray property, and change the intended experiment. A single Fisher–Yates shuffle is $\Theta(n)$, which does not change the overall expected $\Theta(n \log n)$ complexity.

Defence point: shuffling changes the **input distribution seen by quicksort**, not the partition algorithm itself. It protects against adversarial order but adds a constant-factor preprocessing cost.

5.3 Improvement 2: median-of-three pivot selection

For each current subarray `a[lo..hi]` , inspect:

Plain Text

```
first  = a[lo]
middle = a[lo + floor((hi - lo) / 2)]
last   = a[hi]
```

Find which index contains the median value, then exchange that item with `a[lo]` . The existing partition code can then continue unchanged because it always uses `a[lo]` as the

pivot.

Pseudocode:

Plain Text

```
PARTITION(A, lo, hi)
    if useMedianOfThree
        m = MEDIAN_INDEX(A, lo, middle index, hi)
        EXCHANGE(A, lo, m)
    pivot = A[lo]
    perform the original two-scan partition
```

On an already sorted subarray, the first value is smallest, the last is largest, and the middle value is the median. Therefore this directly fixes the particular “first item is the minimum” failure pattern. On less structured data, it samples three candidates and often avoids an extreme pivot.

Median-of-three normally improves the constant factor and makes bad partitions less likely; it does **not** guarantee $\Theta(n \log n)$. An adversarial input can still make the median of these three positions a poor pivot. The extra comparisons and possible exchange also mean that it may be slower on tiny arrays or on input distributions where the baseline was already favourable.

5.4 Improvement 3: insertion sort for small subarrays

Quicksort should stop creating recursive calls once a subarray is small enough:

Plain Text

```
QUICKSORT(A, lo, hi)
    if hi <= lo
        return
    if hi - lo + 1 <= cutoff
        INSERTION-SORT(A, lo, hi)
        return
    j = PARTITION(A, lo, hi)
    QUICKSORT(A, lo, j - 1)
    QUICKSORT(A, j + 1, hi)
```

This does not change the asymptotic complexity by itself. The remaining large partitions are still governed by quicksort, and the small pieces would have required only a bounded amount of work anyway. The benefit is practical: insertion sort has a very small inner loop, avoids recursion and partition overhead, and is efficient when a small subarray is already nearly sorted—which is often true near the leaves of a well-behaved quicksort recursion.

The cutoff must be interpreted consistently. In this implementation the natural condition is “switch when the inclusive subarray length $hi - lo + 1$ is less than or equal to the cutoff.” A cutoff of 0 disables the optimisation. A cutoff of 10 means a subarray of length 10 or less uses insertion sort.

Defence point: this is a hybrid algorithm. It is not claiming insertion sort is asymptotically better; it is exploiting the fact that asymptotics do not determine the fastest method for tiny inputs.

6. Which improvements affect complexity?

Improvement	Effect on complexity	Effect on practical speed
Shuffle once	Changes bad ordered-input behaviour from deterministic $\Theta(n^2)$ to expected $\Theta(n \log n)$; worst-case remains $\Theta(n^2)$	Adds $\Theta(n)$ preprocessing but protects sorted/structured inputs
Median-of-three	Usually improves partition balance but does not guarantee a new asymptotic bound; worst-case remains $\Theta(n^2)$	Often improves constants and fixes sorted-input behaviour for this specific pivot pattern
Insertion cutoff	Does not change quicksort's asymptotic complexity	Reduces recursion and partition overhead on small subarrays

The cleanest answer to the lab's question is therefore:

Shuffling changes the expected behaviour on ordered inputs. Median-of-three improves the balance of typical partitions but does not remove the quadratic worst case in general. The insertion cutoff is primarily a constant-factor optimisation and does not change the asymptotic class.

If the timing table appears to show a complexity change for median-of-three or the cutoff, phrase it carefully: the empirical curve may look better over the tested range, but an asymptotic claim requires reasoning about arbitrarily large inputs, not only the sampled sizes.

7. How to find the insertion cutoff systematically

The cutoff is an empirical machine- and implementation-dependent crossover point. It is the point where the cost of “one more quicksort partition plus recursive calls” becomes larger than the cost of directly insertion-sorting the small subarray.

Method A: benchmark a grid of candidate cutoffs

1. Fix the other choices. For example, evaluate the full hybrid configuration with shuffle and median-of-three enabled.
2. Select a representative set of input sizes, preferably including 1000 , 3000 , 10000 , and 30000 .
3. Test a coarse grid such as 0, 5, 10, 15, 20, 30, 40, 50, 60, 80 .
4. Run every cutoff against the same generated inputs and the same JVM conditions.
5. Repeat each measurement and compare the median or minimum stable result rather than one noisy observation.
6. Identify the neighbourhood of the fastest value.
7. Refine around it, for example testing 35, 40, 45, 50, 55 if the first scan points near 45.
8. Accept a cutoff only if nearby values on **both** sides are worse or no better in a consistent way.
9. Round the answer to a practical value such as the nearest 5 or 10, as the handout permits.

A cutoff-selection table should look like this:

Cutoff	Random	95% sorted	Sorted	Mean/aggrega te	Interpretation
0	...	...	...	...	Baseline for the hybrid configuration
10	...	...	...	...	Candidate
20	...	...	...	...	Candidate
40	...	...	...	...	Candidate
60	...	...	...	...	Candidate
80	...	...	...	...	Candidate; may be too high

Do not select the cutoff solely because it is fastest on one input size or one input category. The best choice is the lowest robust aggregate time across the workload the lab cares about.

Method B: crossover explanation without relying on a table

There is another way to explain the same method:

Imagine the recursion tree. Near the top, subarrays are large, so partitioning is worthwhile. Near the leaves, subarrays are tiny, so the fixed costs of a recursive call, pivot setup, scans, and exchanges dominate. The cutoff is where these two strategies cross. I locate that crossing experimentally by increasing the cutoff until the time falls, then continuing until making the cutoff larger starts to increase time again.

This explains why the curve is often U-shaped: a cutoff that is too small leaves too much recursion; a cutoff that is too large makes insertion sort handle arrays where quicksort would have been cheaper.

Evidence from the supplementary reconstruction

A controlled supplementary run of the repository's intended algorithms was used to sanity-check the direction of the effects. It confirmed the qualitative points: the baseline first-element quicksort deteriorated sharply on sorted input; shuffling removed that ordered-input explosion; median-of-three handled the sorted case particularly well; and cutoff values should be compared as part of a complete hybrid configuration rather than inferred from cutoff-only tests.

The exact numbers should not be copied into a formal submission unless they are regenerated with the official `Bench.java` on the submission machine. Timing values are machine-dependent. The correct coursework evidence is an `output.txt` generated from the completed `Bench.java`, with the exact configurations and cutoff stated beside each report.

8. How to determine the best combination

The three binary improvements yield eight structural combinations, before varying the cutoff. A defensible experiment includes at least:

Combination family	Example configuration
No improvements	<code>(false, false, 0)</code>
Each single improvement	<code>(true, false, 0)</code> , <code>(false, true, 0)</code> , <code>(false, false, c)</code>
Each pair	<code>(true, true, 0)</code> , <code>(true, false, c)</code> , <code>(false, true, c)</code>

All three

(true, true, c)

For each configuration, record all three input categories. Keep the baseline line in `Bench.java` because the handout requests it. Give each report a descriptive name, such as:

Java

```
executionTimeReport("Quick baseline", new Quick(false, false, 0)::sort);
executionTimeReport("Quick shuffle", new Quick(true, false, 0)::sort);
executionTimeReport("Quick median-of-three", new Quick(false, true, 0)::sort);
executionTimeReport("Quick all improvements cutoff 40", new Quick(true, true, 40)::sort);
```

The “best” combination is not simply the one with the lowest time on one row. It is the configuration with the best stable performance across the intended input categories and sizes, while avoiding a complexity failure on sorted data. In most standard quicksort implementations, a strong practical choice is **shuffle + median-of-three + a moderate insertion cutoff**, but the correct cutoff and even the best pair are empirical for this code, JVM, and hardware.

A good conclusion has this form:

“The selected configuration was (true, true, c), where c was chosen by a coarse-to-fine sweep. It was preferred because it avoided the deterministic sorted-input failure, generally produced better partitions, and reduced leaf-level recursion. The choice was based on repeated measurements across random, 95%-sorted, and sorted arrays, not on one timing observation. The cutoff values immediately below and above c were slower, so c was a stable practical choice.”

If the data instead shows that shuffle plus cutoff is faster than all three improvements, report that honestly. An optimisation is not automatically beneficial merely because it is theoretically sensible: median-of-three has overhead, and shuffling has a full-array cost.

9. Important implementation and debugging details

9.1 Shuffle exactly once

The call to `shuffle(a)` belongs in the public `sort` method, before `sort(a, 0, a.length - 1)`. It must not be placed in the recursive helper. The repository already supplies a Fisher–Yates-style `shuffle` method and a fixed random seed, which supports repeatable experiments.

9.2 Median-of-three must exchange the selected value into `a[lo]`

Merely computing the median index is not enough because the existing partition code reads `a[lo]` as the pivot. The selected median must be exchanged with the first element before `pivot = a[lo]` is executed.

9.3 Use inclusive subarray lengths

The subarray `a[lo..hi]` contains `hi - lo + 1` values. A frequent off-by-one bug is to use `hi - lo` as the size. The cutoff condition should be tested against the inclusive length.

9.4 Preserve duplicate-key safety

The partition loops deliberately use careful inequalities and boundary checks. Do not casually replace `<` by `<=` or remove the checks around `i == hi` and `j == lo + 1`. Equal values can otherwise cause a pointer not to advance, an infinite loop, or an invalid partition. The correctness test includes generated duplicate values.

9.5 Test correctness before trusting timing

A faster wrong algorithm is not an optimisation. Always run the correctness section first. The supplied harness compares against `Arrays.sort` and tests thousands of generated arrays up to size 1000. For debugging, add small targeted cases such as:

Plain Text

```
{}  
{1}  
{2, 1}  
{1, 2}  
{2, 2, 2}  
{3, 1, 2, 1}  
{5, 4, 3, 2, 1}
```

10. Likely defence questions and model defences

Q1. Why is insertion sort quadratic on random input?

Because each new item may have to move through a growing sorted prefix. The total number of comparisons/shifts is approximately the sum of the prefix lengths, `1 + 2 + ... + (n - 1)`, which is $\Theta(n^2)$.

Q2. Why is insertion sort linear on sorted input?

Every new item is already at least as large as the previous item, so the inner shifting loop

does no work. The algorithm still scans the array once, giving $\Theta(n)$.

Q3. Why does 95% sorted not automatically mean linear insertion sort?

Because runtime is controlled by inversions, not by the label alone. A small fraction of random values can create many long-distance shifts. The actual timing trend must be inspected, and the answer should be based on the benchmark's sampled family.

Q4. Why is merge sort still linearithmic on sorted input?

The supplied code does not detect that the halves are already in global order. It recursively divides and then copies/merges at every level. An optimisation such as “skip merge if `a[mid] <= a[mid+1]`” could make the sorted case linear, but that optimisation is not present in this repository.

Q5. Why is quicksort fast on random data but poor on sorted data here?

The pivot is always the first element. Random input usually gives a non-extreme first element, while sorted input makes it the minimum. The resulting partitions are balanced on average in the first case and maximally unbalanced in the second.

Q6. Does shuffling guarantee $O(n \log n)$ quicksort?

No. It gives expected $O(n \log n)$ behaviour by making the input order random relative to the pivot rule. The worst-case sequence still exists, although its probability becomes very small.

Q7. Does median-of-three guarantee balanced partitions?

No. It is only a sample of three positions. It often avoids extreme pivots and fixes a sorted subarray for first/middle/last sampling, but an adversarial arrangement can still make the selected median poor.

Q8. Why can median-of-three make a small input slower?

It adds comparisons and possibly an exchange before partitioning. On a tiny subarray, that overhead can be larger than the saving from a better partition. This is one reason for a small-subarray cutoff.

Q9. Why is insertion sort suitable at the leaves of quicksort?

The leaves are small, and insertion sort has low constant overhead, no recursive calls, and a short inner loop. The small pieces are also often nearly ordered because of the preceding

partitions.

Q10. Why does the cutoff not change the asymptotic complexity?

Changing a fixed threshold replaces the treatment of bounded-size subproblems. The large-subarray recursion still determines the asymptotic growth. The cutoff improves constants, cache behaviour, and call overhead.

Q11. Why not use the largest possible cutoff if insertion sort is simple?

Insertion sort is quadratic in the size of the subarray. A high cutoff makes it process medium-sized pieces quadratically, eventually costing more than quicksort partitioning would have cost. The experiment should show performance worsening on both sides of the selected crossover.

Q12. Why must the baseline be retained in `Bench.java` ?

It provides a control group. Without the unchanged `(false, false, 0)` result, there is no fair reference for claiming that an improvement helped.

Q13. Why are raw benchmark times not proof of complexity?

They are finite, noisy samples affected by hardware and runtime effects. They provide empirical evidence of a growth trend. Complexity claims are strengthened by combining the trend with the algorithm's recurrence or loop structure.

Q14. What extra space does each algorithm use?

Insertion sort is in-place and uses $\Theta(1)$ auxiliary storage. The supplied merge sort uses an auxiliary array of $\Theta(n)$. Quicksort partitions in-place but uses recursion stack space: expected $\Theta(\log n)$ for balanced recursion and $\Theta(n)$ in a worst-case chain.

Q15. Is quicksort stable in this implementation?

No stability guarantee is provided. Partition exchanges can reorder equal keys. Since the repository sorts primitive integers, stability is not observable as an association with records, but it remains an algorithmic property.

Q16. What would you improve beyond the required lab?

Possible extensions include three-way partitioning for many duplicate keys, tail-recursion elimination or recursing on the smaller partition first to limit stack depth, an introspective

fallback to heapsort, and the merge-sort optimisation that skips a merge when `a[mid] <= a[mid+1]` . These are extensions, not required claims about the current repository.

11. Recommended final submission checklist

Before submission, ensure that:

1. `Quick.java` contains working implementations of shuffle, median-of-three exchange, and insertion cutoff.
2. The cutoff condition uses the correct inclusive subarray size.
3. `Bench.java` benchmarks the baseline and the selected improved configuration(s).
4. The correctness tests pass before timing results are accepted.
5. `output.txt` is generated from the final code on the intended machine.
6. `answers.txt` records the complexity classifications, the complexity effect of each improvement, the systematic cutoff method, and the best combination.
7. The written explanation distinguishes isolated tests from cumulative combination tests.
8. Timing claims identify the machine/configuration and do not present supplementary measurements as official repository output.
9. Known limitations are disclosed, especially the machine dependence of exact cutoff values and benchmark times.

12. Final one-minute oral summary

“Lab 1 compares growth patterns. Insertion sort is linear when the input has few inversions, but quadratic when many items must shift. Merge sort repeatedly divides and merges, so this implementation is linearithmic regardless of input order. Basic quicksort is linearithmic on average when partitions are reasonably balanced, but the first-element pivot makes sorted data a quadratic worst case. Lab 2 tests three practical improvements. Shuffling once prevents original order from systematically producing bad pivots and gives expected linearithmic behaviour. Median-of-three chooses a better pivot candidate, especially fixing the sorted first/middle/last pattern, but does not mathematically remove the worst case. An insertion cutoff uses a low-overhead quadratic method only on tiny subarrays; it improves constants rather than asymptotic complexity. I measured each improvement against the same baseline, then measured combinations cumulatively. I selected the cutoff through a coarse-to-fine sweep and checked that nearby values were worse across the three input categories.”

References

- [1] [MubiruEltonFelix1/dsa-coursework-year-two](#) coursework repository
- [2] Algorithms, 4th Edition: Quicksort
- [3] Algorithms, 4th Edition: Mergesort
- [4] Algorithms, 4th Edition: Elementary Sorting